

Lecture 7: PHÂN ĐOẠN ẢNH (Image Segmentation)

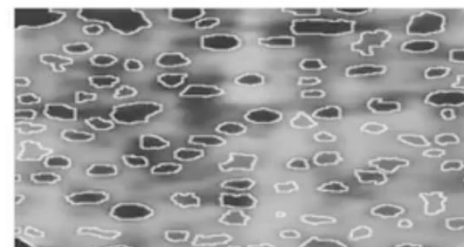
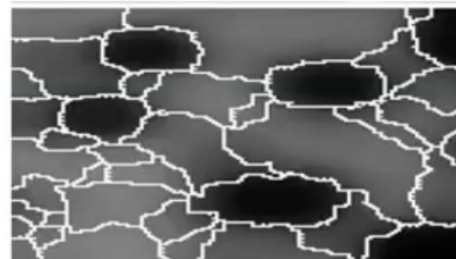
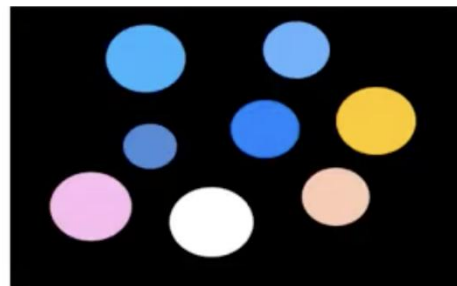
Nội dung

1. Khái niệm phân đoạn ảnh
2. Giới thiệu về bài toán phân đoạn ảnh
3. Approaches in Image Segmentation
4. Image Segmentation Techniques
 - Threshold Based Segmentation
 - Edge Based Segmentation
 - Region-Based Segmentation
 - **Clustering Based Segmentation**
 - **Artificial Neural Network Based Segmentation**

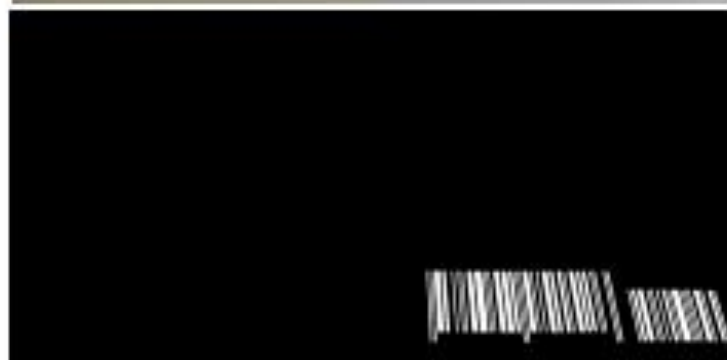
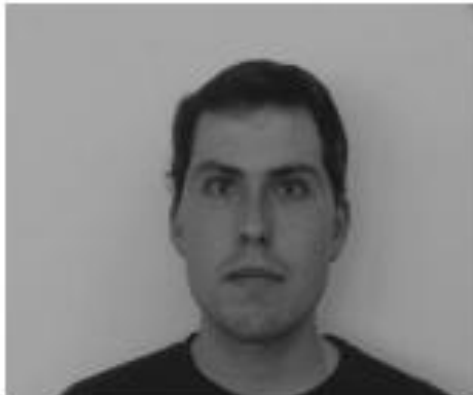
Khái niệm phân đoạn ảnh

- Cho R biểu diễn vùng không gian ảnh $\rightarrow$ Phân đoạn ảnh là phân chia R thành n vùng con: $R_1, R_2, \dots, R_n$
- Sao cho:
 - $\bigcup_{i=1}^n R_i = R$
 - R_i : Vùng ảnh chứa tập các pixel được kết nối, $i = 0, 1, 2, \dots, n$
 - $R_i \cap R_j = \emptyset$ với $i \neq j$
 - $Q(R_i) = \text{True}$: nghĩa là các pixel trong vùng R_i có tính chất giống nhau
 - $Q(R_i \cup R_j) = \text{False}$: nghĩa là các pixel ở 2 vùng R_i và R_j có tính chất khác nhau
- Lưu ý:
 - Hai vùng R_i và R_j được cho là liền kề nếu sự kết hợp của chúng tạo thành một vùng kết nối
 - Nếu vùng được hình thành bởi sự kết hợp của 2 vùng không được kết nối $\rightarrow$ 2 vùng được gọi là rời nhau.

Ví dụ phân đoạn ảnh



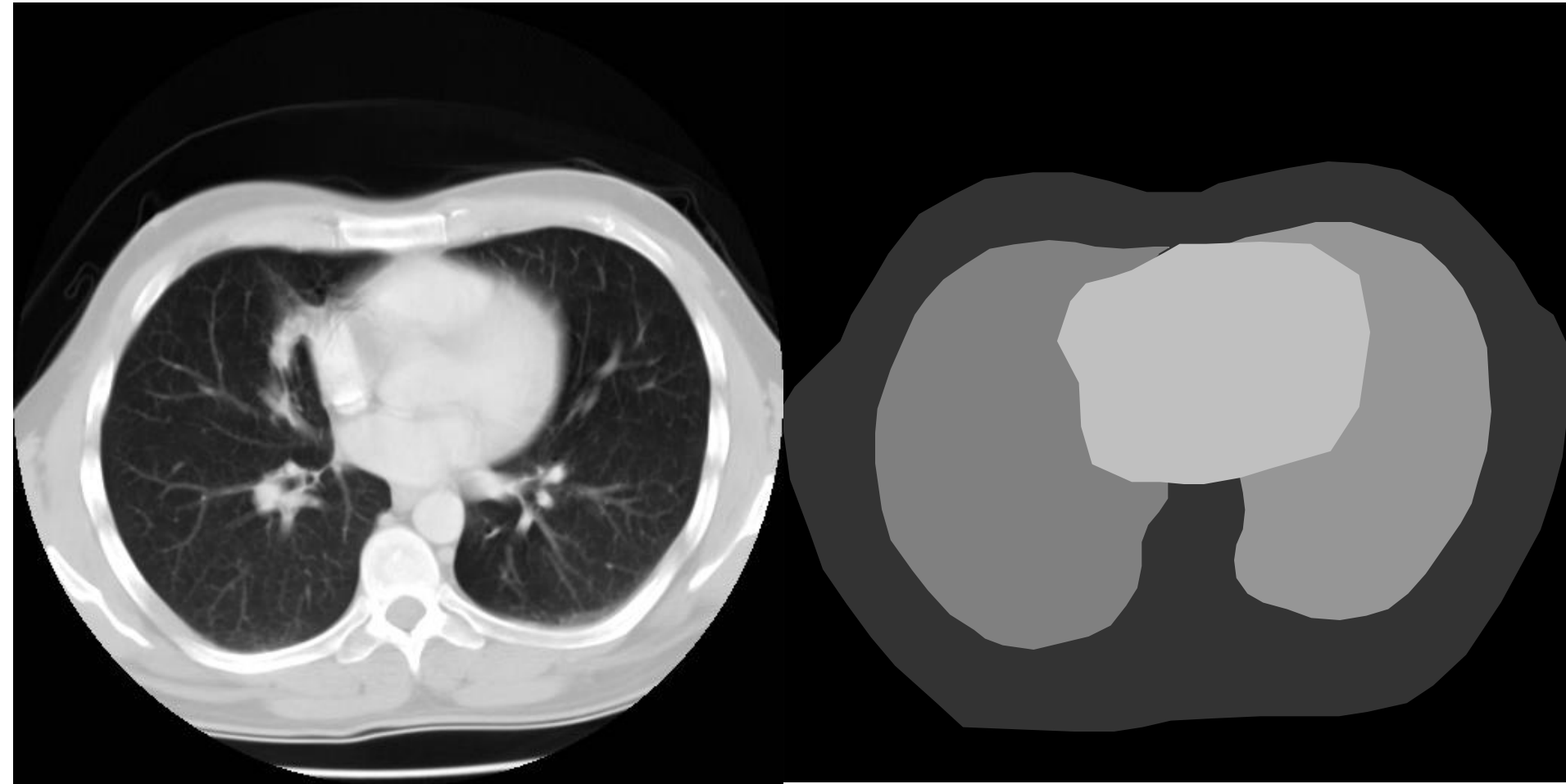
Phân đoạn ảnh - ứng dụng



Phân đoạn ảnh - ứng dụng



Phân đoạn ảnh - ứng dụng



Phân đoạn ảnh - ứng dụng

ponents or broken connection paths. There is no point past the level of detail required to identify those components.

Segmentation of nontrivial images is one of the most difficult tasks in image processing. Segmentation accuracy determines the effectiveness of computerized analysis procedures. For this reason, considerable effort can be taken to improve the probability of rugged segmentation. In applications such as industrial inspection applications, at least some degree of ruggedness in the environment is possible at times. The experienced image designer invariably pays considerable attention to such

ponents or broken connection paths. There is no point past the level of detail required to identify those components.

Segmentation of nontrivial images is one of the most difficult tasks in image processing. Segmentation accuracy determines the effectiveness of computerized analysis procedures. For this reason, considerable effort can be taken to improve the probability of rugged segmentation. In applications such as industrial inspection applications, at least some degree of ruggedness in the environment is possible at times. The experienced image designer invariably pays considerable attention to such

Giới thiệu về bài toán phân đoạn ảnh

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

**Classification
+ Localization**



CAT

Single Object

**Object
Detection**



DOG, DOG, CAT

Multiple Object

**Instance
Segmentation**



DOG, DOG, CAT

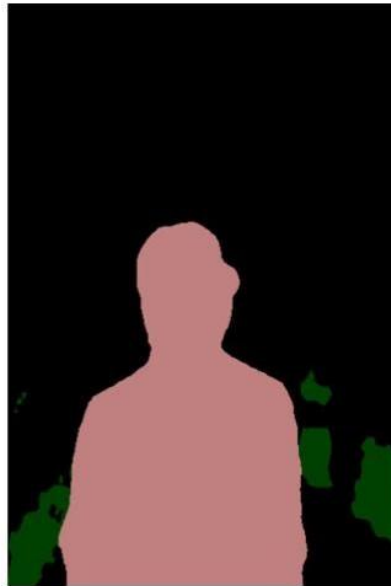
[This image is CC0 public domain](#)

Phân đoạn ảnh - Image Segmentation

- Mục đích: xác định các vùng điểm ảnh liên quan nhau
- Không phân biệt các đối tượng cùng lớp trong ảnh



Original Image



Semantic Segmentation



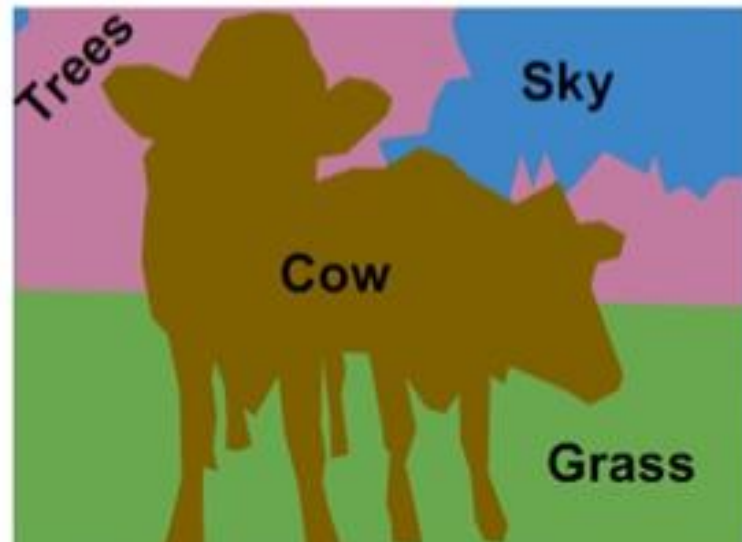
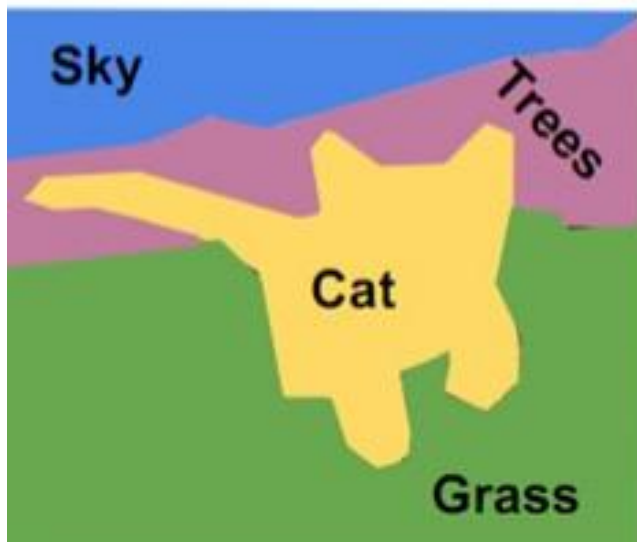
Instance Segmentation

- Là bài toán kinh điển của thị giác máy tính

Image Segmentation

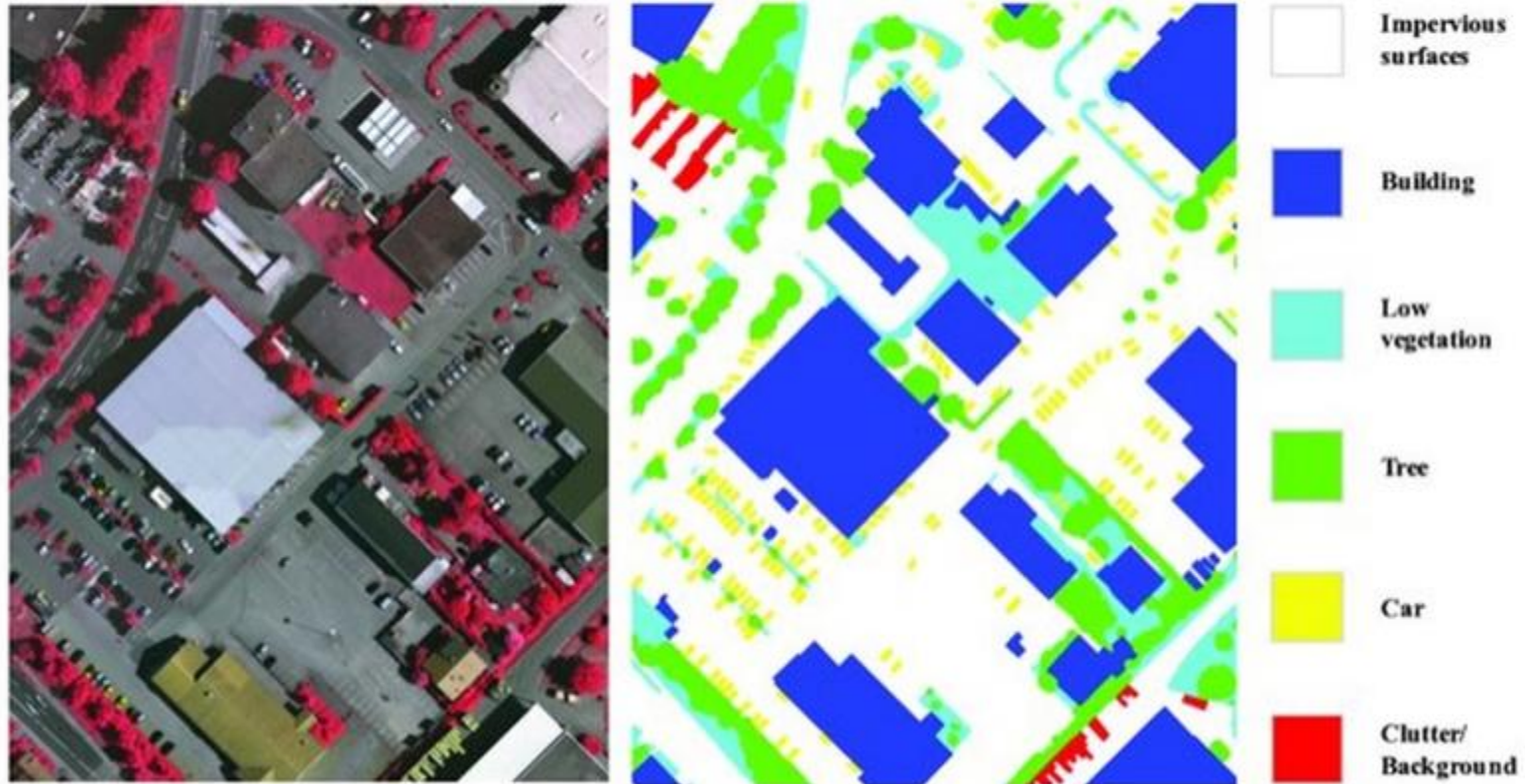


This image is CC0 public domain



Một số ứng dụng phân đoạn ảnh

- Phân đoạn ảnh vệ tinh và hàng không



Một số ứng dụng phân đoạn ảnh

- Xe tự hành



Một số ứng dụng phân đoạn ảnh

- Y tế

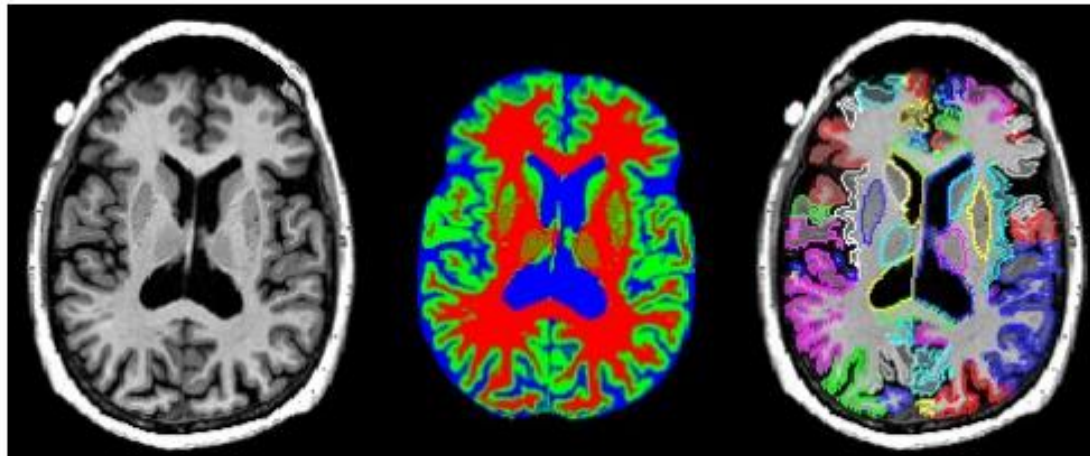
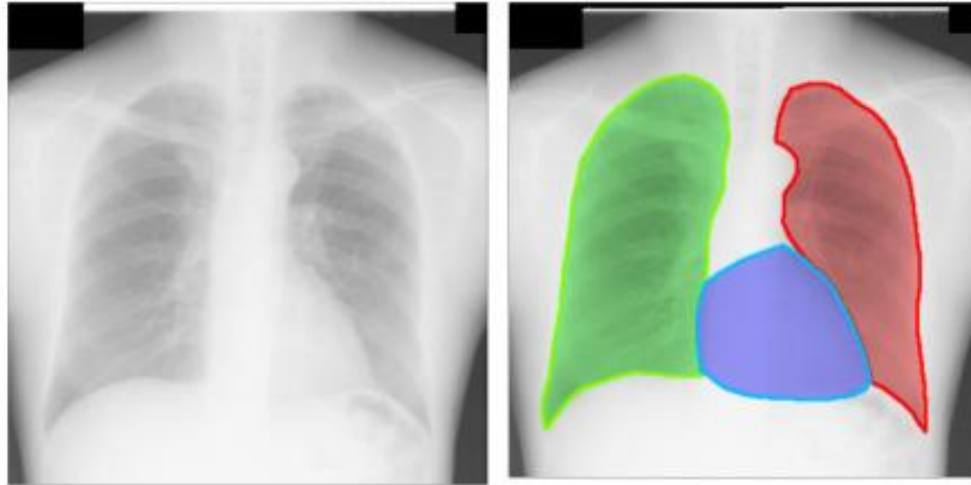
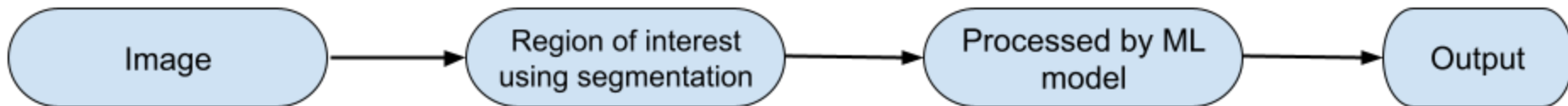


Image Segmentation

- Phân đoạn hình ảnh giúp giảm độ phức tạp của hình ảnh để làm cho quá trình xử lý hoặc phân tích hình ảnh sau đó trở nên đơn giản hơn.
- Nói một cách dễ hiểu, phân đoạn là gán nhãn cho pixel



Approaches in Image Segmentation

1. Similarity approach:

- Dựa trên việc phát hiện sự tương đồng giữa các pixel để tạo thành một vùng.
- Các thuật toán ML như phân cụm dựa trên cách tiếp cận này

2. Discontinuity approach:

- Dựa trên sự gián đoạn của các giá trị cường độ pixel.
- Các kỹ thuật Phát hiện Đường, Điểm và Cạnh sử dụng cách tiếp cận này để thu được kết quả phân đoạn trung gian có thể được xử lý sau đó để thu được hình ảnh được phân đoạn cuối cùng.

Approaches in Image Segmentation

- *Không có kỹ thuật phân đoạn ảnh thích hợp để áp dụng cho tất cả các ảnh.*

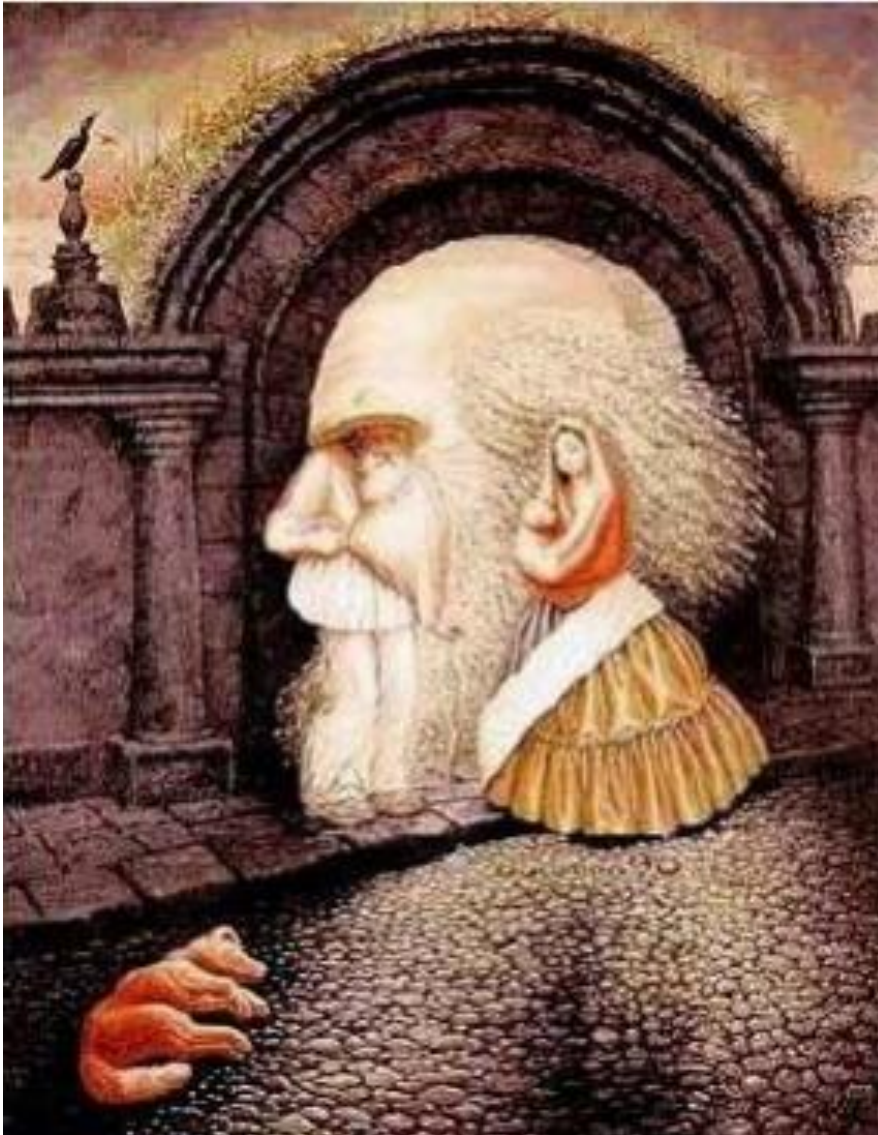
Một số khó khăn

- Điều kiện chiếu sáng
- Đối tượng che khuất
- Ảnh nhiễu/chất lượng thấp



- Chưa có thuật toán phân đoạn là tốt nhất cho mọi loại ảnh

Example - bằng mắt thường ???

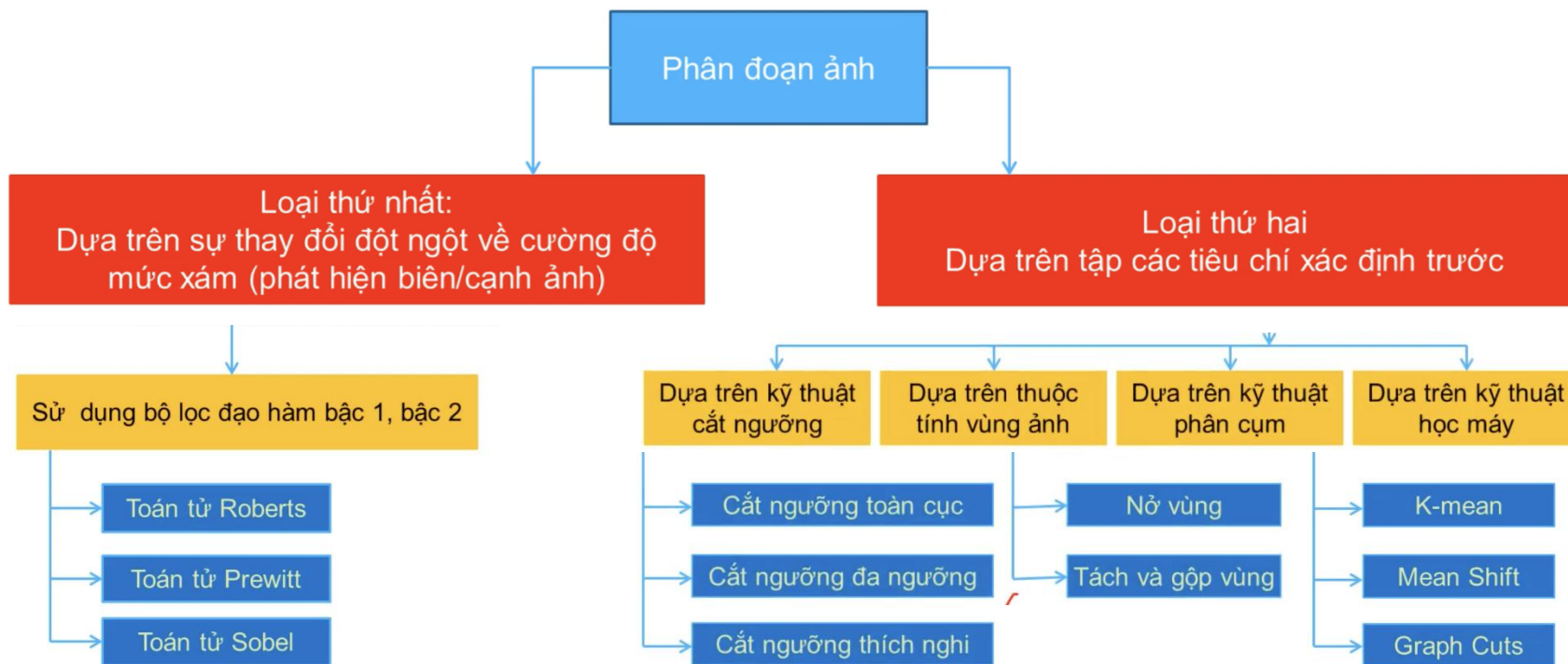


- Ông già và các thứ khác ?
- Hai người và con chó ?

Challenge



Phân loại phân đoạn ảnh



Mục đích của phân đoạn ảnh

- **Mục đích của phân đoạn ảnh:**
 - Hiểu ảnh
 - Đối sánh và nhận dạng ảnh
 - Dò tìm đối tượng trong ảnh
 - Phân lớp
 - Tăng tốc xử lý các bài toán liên quan tới xử lý vùng ảnh
 - ...

CÁC KỸ THUẬT PHÂN ĐOẠN TRUYỀN THỐNG

Phân đoạn ảnh

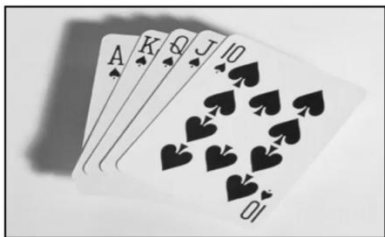
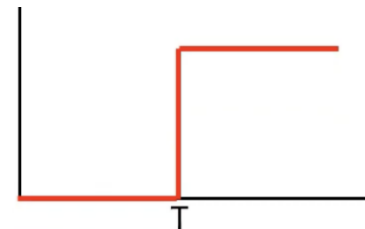
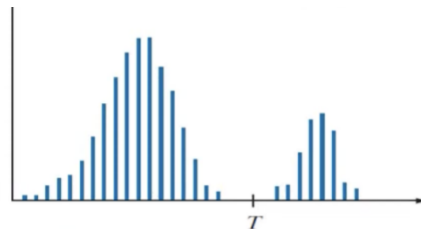
- Bảng kỹ thuật cắt ngưỡng -



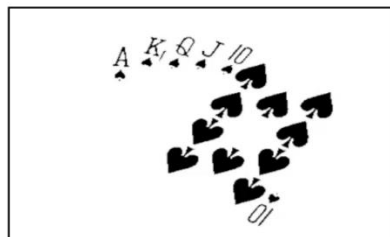
Phân đoạn ảnh dựa vào ngưỡng toàn cục

- Cắt ngưỡng toàn cục (đơn ngưỡng):** cắt ngưỡng dựa vào 1 giá trị ngưỡng T cho toàn bộ ảnh

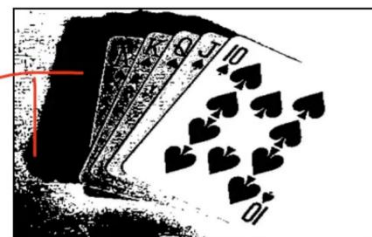
$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases} \quad (1)$$



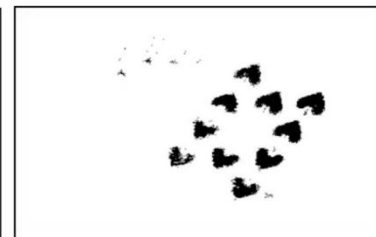
Ảnh gốc



Ảnh sau khi
cắt ngưỡng



Ngưỡng
quá cao



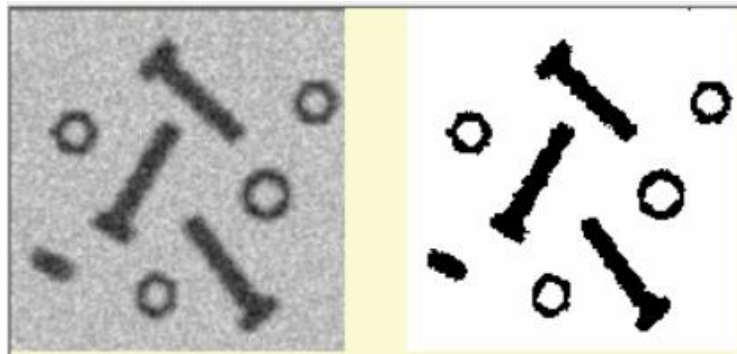
Ngưỡng
quá thấp

Phân đoạn ảnh dựa vào ngưỡng toàn cục

- Thông thường ảnh kết quả ở dạng nhị phân (0 ứng với nền, 1 ứng với vật hoặc ngược lại)
- Vấn đề là: ***Làm thế nào để tìm được ngưỡng T ?***
- Có hai cách cơ bản là:
 - Ngưỡng cố định
 - **Tìm ngưỡng tự động dựa vào histogram: OTSU**

Phân đoạn ảnh dựa vào ngưỡng toàn cục

- Ngưỡng cố định
 - Tìm được do thống kê
 - Do người dùng cung cấp
 - Ví dụ:



Phân đoạn dựa trên ngưỡng TOÀN CỤC

- INPUT: ảnh xám
- OUTPUT: ảnh nhị phân
- Mục đích: tách được foreground và background.
- Thực hiện bằng cách chọn ngưỡng T , và tạo ảnh output :

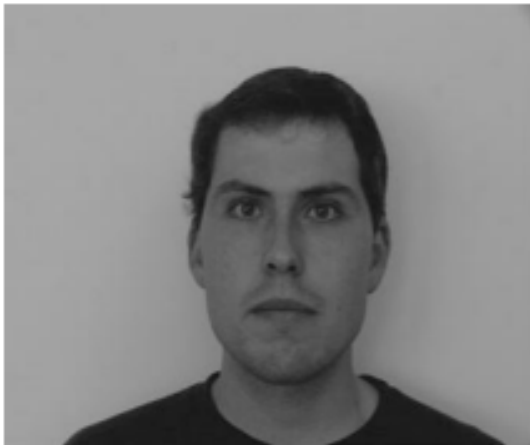
$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

Algorithm 6.1: Basic thresholding

1. Search all the pixels $f(i, j)$ of the image f . An image element $g(i, j)$ of the segmented image is an object pixel if $f(i, j) \geq T$, and is a background pixel otherwise.

Gray-level thresholding

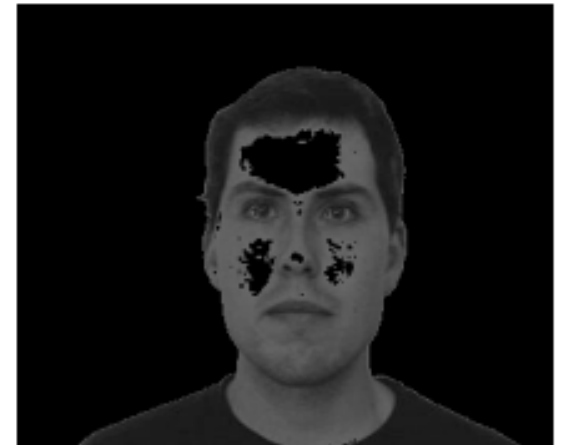
How can holes be filled?



Original image
Peter $f[x,y]$

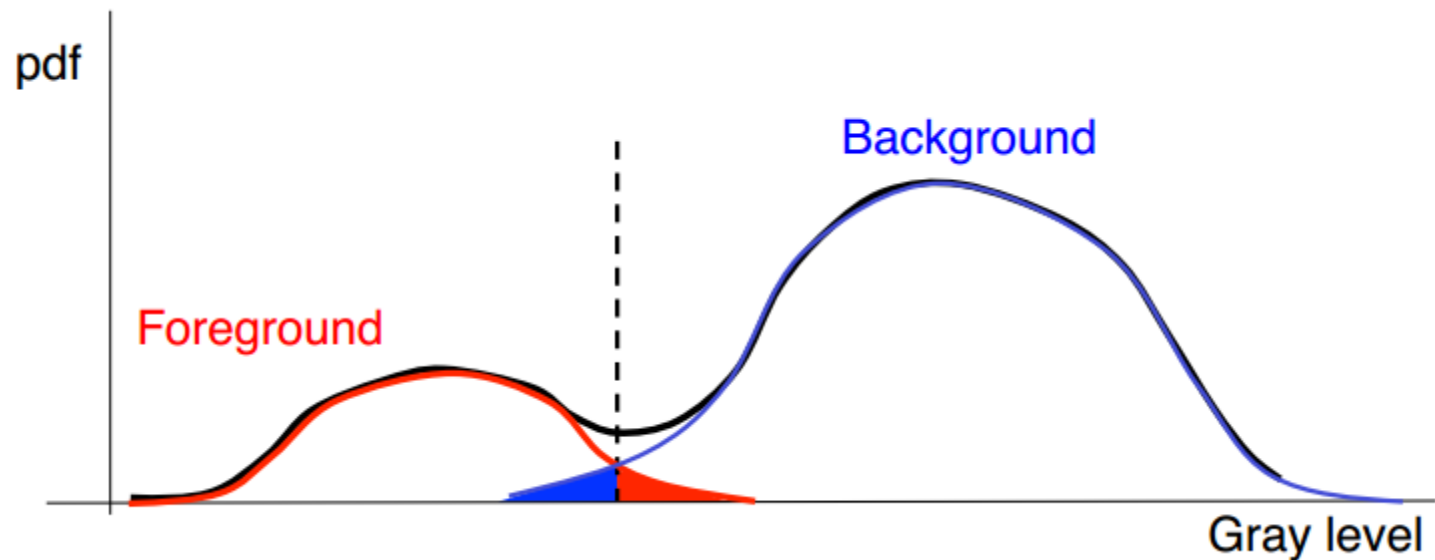


Thresholded
Peter $m[x,y]$



$$f[x,y] \cdot m[x,y]$$

How to choose the threshold?

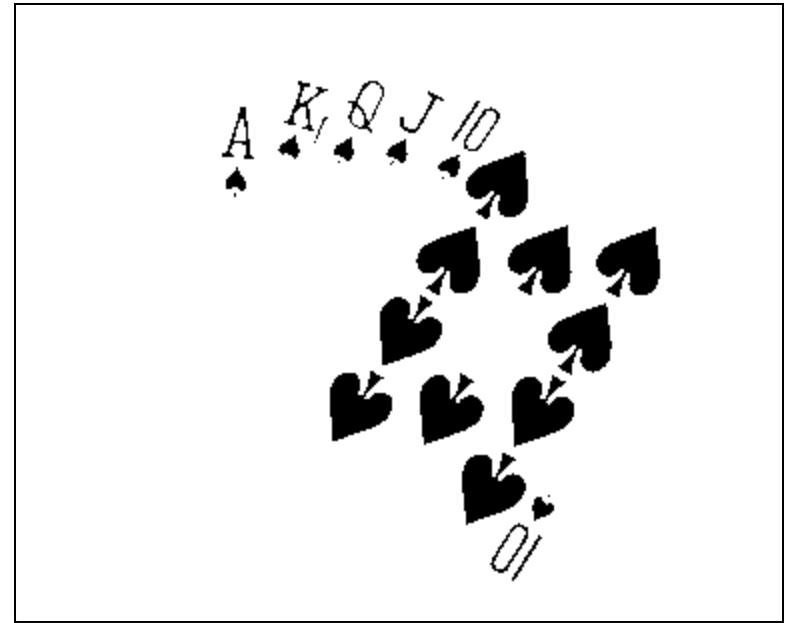


Thresholding Example

- Imagine a poker playing robot that needs to visually interpret the cards in its hand



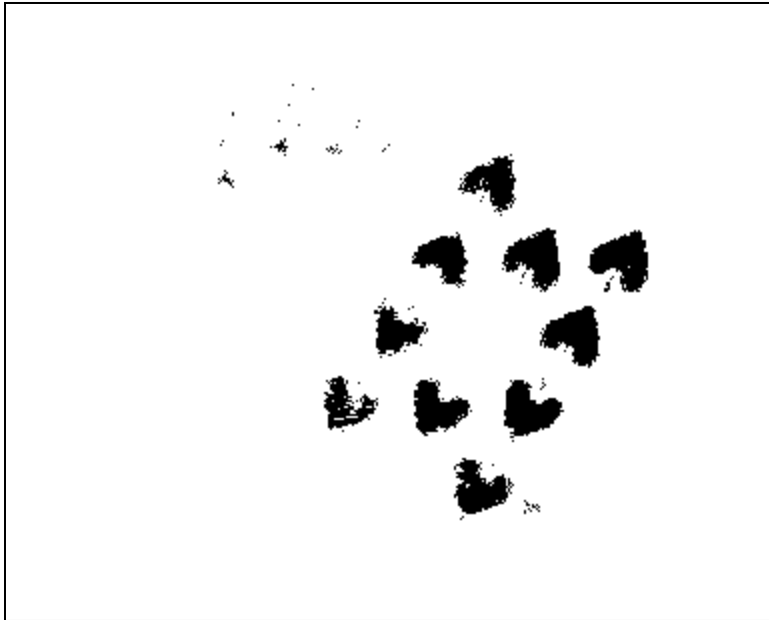
Original Image



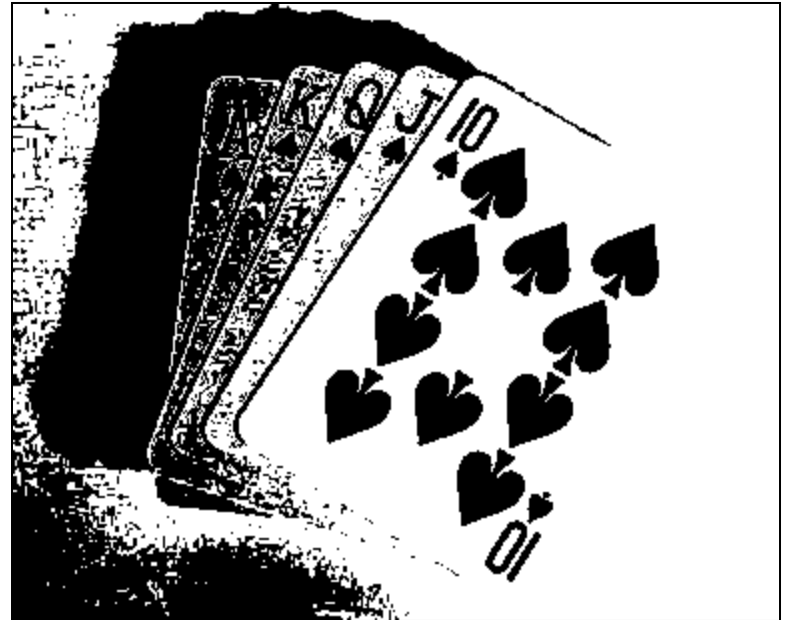
Thresholded Image

But Be Careful

- Nếu chọn ngưỡng sai kết quả có thể là thảm họa ???



Threshold Too Low



Threshold Too High

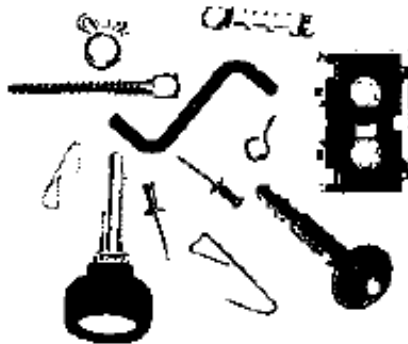
Thresholding Example



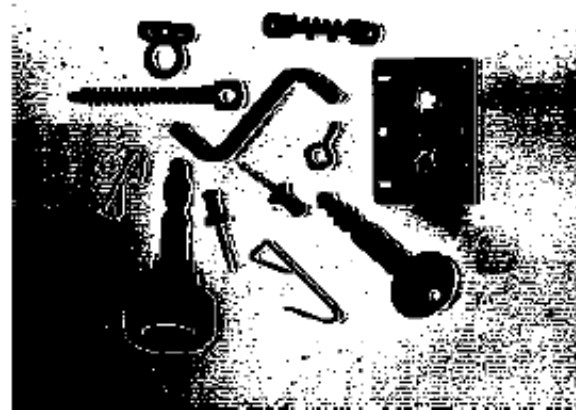
(a)



(b)



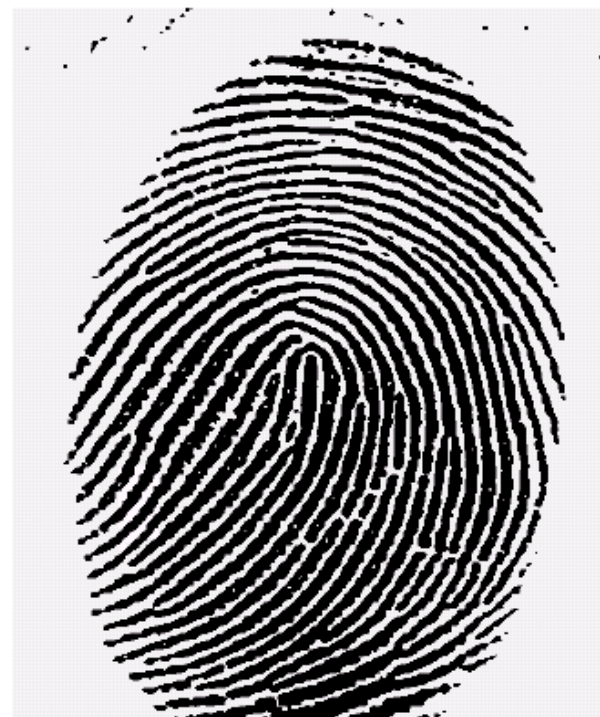
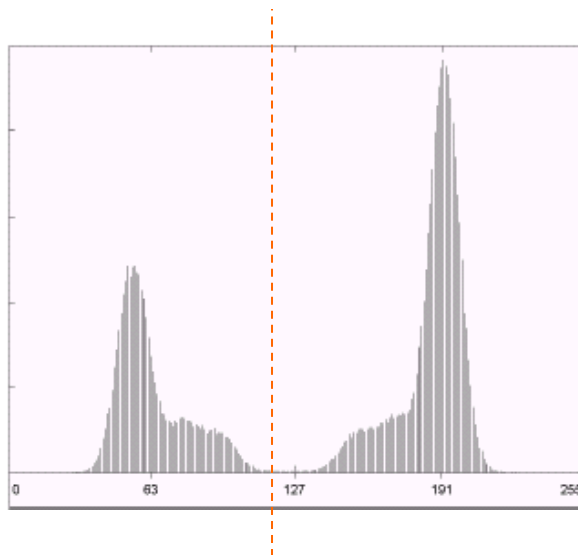
(c)



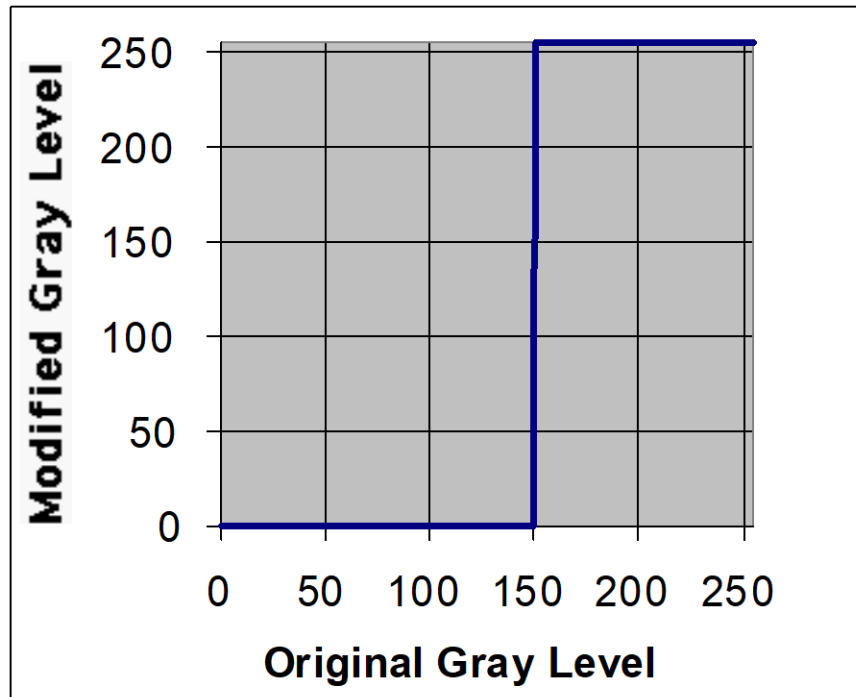
(d)

Figure 6.1: Image thresholding. (a) Original image. (b) Threshold segmentation. (c) Threshold too low. (d) Threshold too high.

Thresholding Example



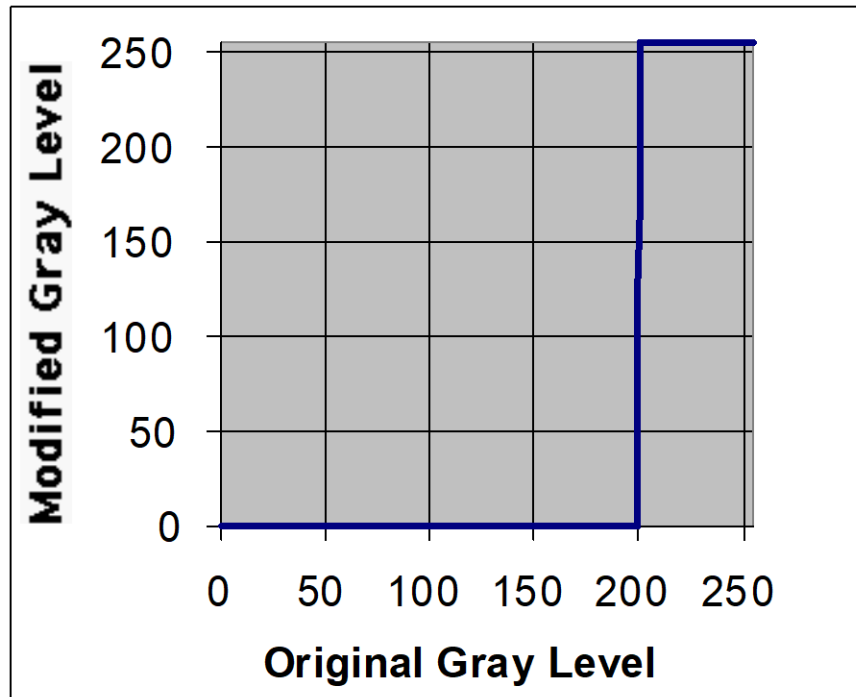
Gray-Level Thresholding



level= 150



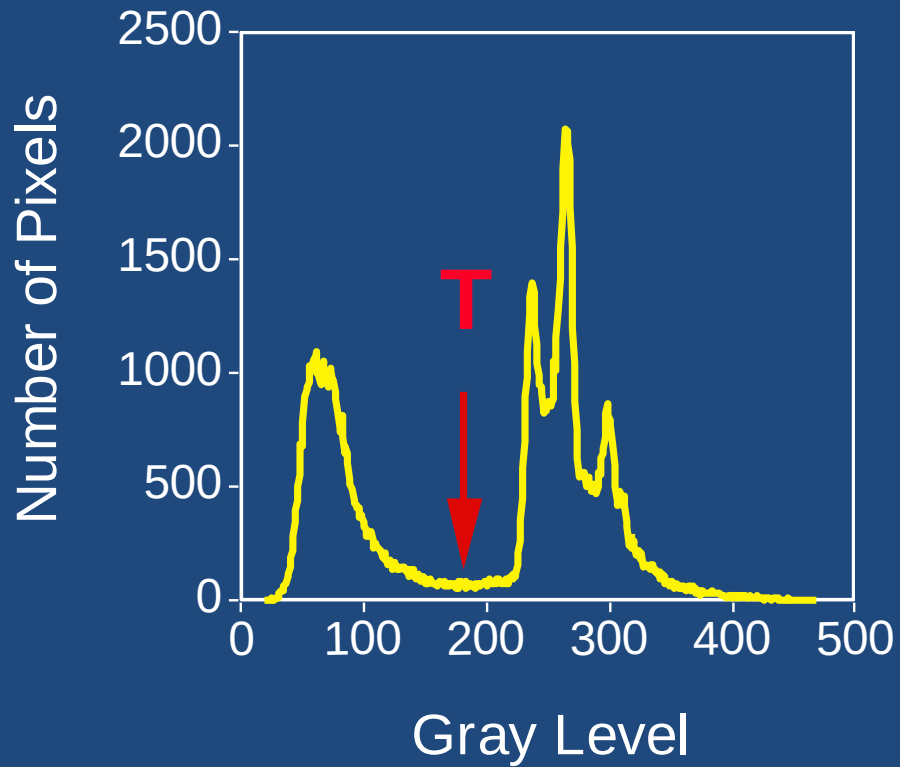
Gray-Level Thresholding



level= 200



Gray-Level Thresholding



Gray-Level Thresholding

- Gray-level thresholding transforms: biến đổi ảnh đầu vào với kết quả đầu ra tương ứng

$$g(i, j) = \begin{cases} 1 & \text{for } f(i, j) \geq T \\ 0 & \text{for } f(i, j) < T \end{cases}$$

OR

$$g(i, j) = \begin{cases} 1 & \text{for } f(i, j) < T \\ 0 & \text{for } f(i, j) \geq T \end{cases}$$

OR

$$g(i, j) = \begin{cases} f(i, j) & \text{for } f(i, j) \geq T \\ 0 & \text{for } f(i, j) < T \end{cases}$$

OR

$$g(i, j) = \begin{cases} f(i, j) & \text{for } f(i, j) < T \\ 0 & \text{for } f(i, j) \geq T \end{cases}$$

Gray-Level Thresholding

3	5	7	3	4	2	1
2	4	9	10	22	9	3
3	5	12	11	15	10	3
5	6	11	9	17	19	1
2	3	11	12	18	16	2
3	6	8	10	18	9	5
4	6	7	8	3	3	1

3	5	7	3	4	2	1
2	4	9	10	22	9	3
3	5	12	11	15	10	3
5	6	11	9	17	19	1
2	3	11	12	18	16	2
3	6	8	10	18	9	5
4	6	7	8	3	3	1

threshold $T \geq 10$

3	5	7	3	4	2	1
2	4	9	10	22	9	3
3	5	12	11	15	10	3
5	6	11	9	17	19	1
2	3	11	12	18	16	2
3	6	8	10	18	9	5
4	6	7	8	3	3	1

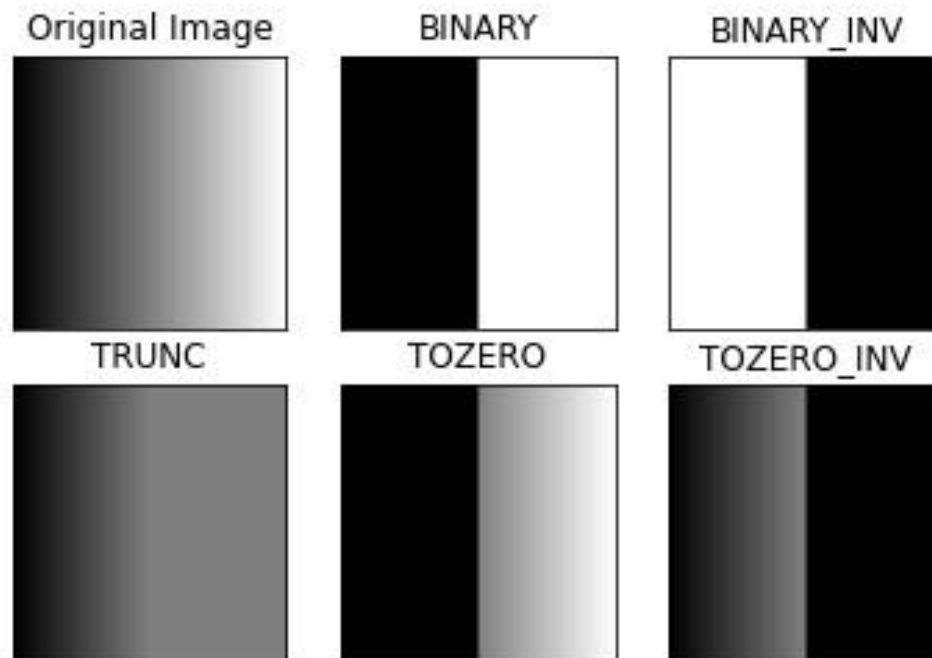
threshold $T \geq 11$

3	5	7	3	4	2	1
2	4	9	10	22	9	3
3	5	12	11	15	10	3
5	6	11	9	17	19	1
2	3	11	12	18	16	2
3	6	8	10	18	9	5
4	6	7	8	3	3	1

threshold $T \geq 12$

Simple Thresholding

```
thresh1 = cv.threshold(img,127,255,cv.THRESH_BINARY)  
thresh2 = cv.threshold(img,127,255,cv.THRESH_BINARY_INV)  
thresh3 = cv.threshold(img,127,255,cv.THRESH_TRUNC)  
thresh4 = cv.threshold(img,127,255,cv.THRESH_TOZERO)  
thresh5 = cv.threshold(img,127,255,cv.THRESH_TOZERO_INV)
```



CODE

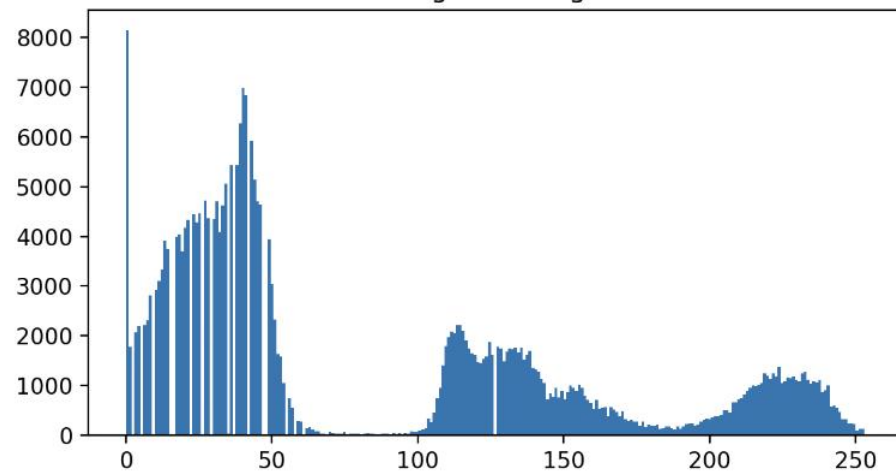
```
def phan_doan_bang_cat_nguong(img,nguong): # Định nghĩa hàm phân đoạn bằng cắt ngưỡng
    m, n = img.shape
    img_phan_doan_cat_nguong = np.zeros([m, n])
    for i in range(m):
        for j in range(n):
            if (img[i,j] < nguong):
                img_phan_doan_cat_nguong[i,j] = 0
            else:
                img_phan_doan_cat_nguong[i,j] = 225 # tương đương gt 1 trong công thức 1
    return img_phan_doan_cat_nguong
```

Kết quả

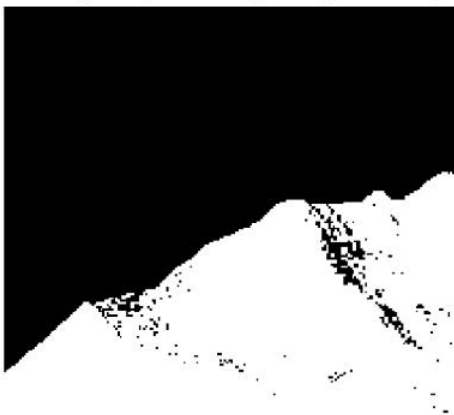
Ảnh gốc



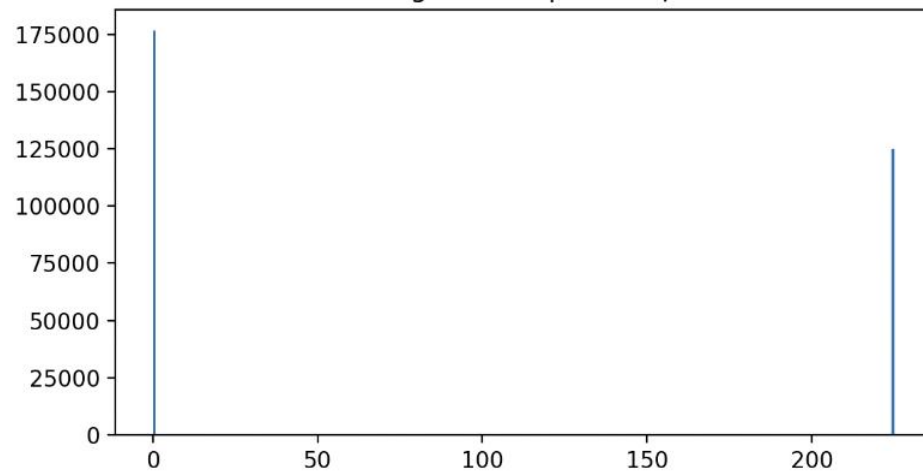
Hitogram ảnh gốc



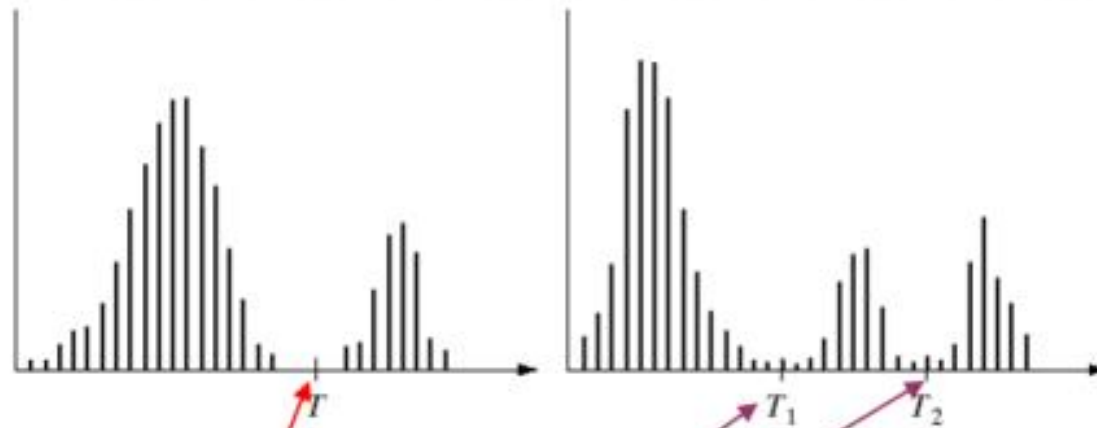
Ảnh phân đoạn dựa vào ngưỡng toàn cục = 110



Hitogram ảnh phân đoạn

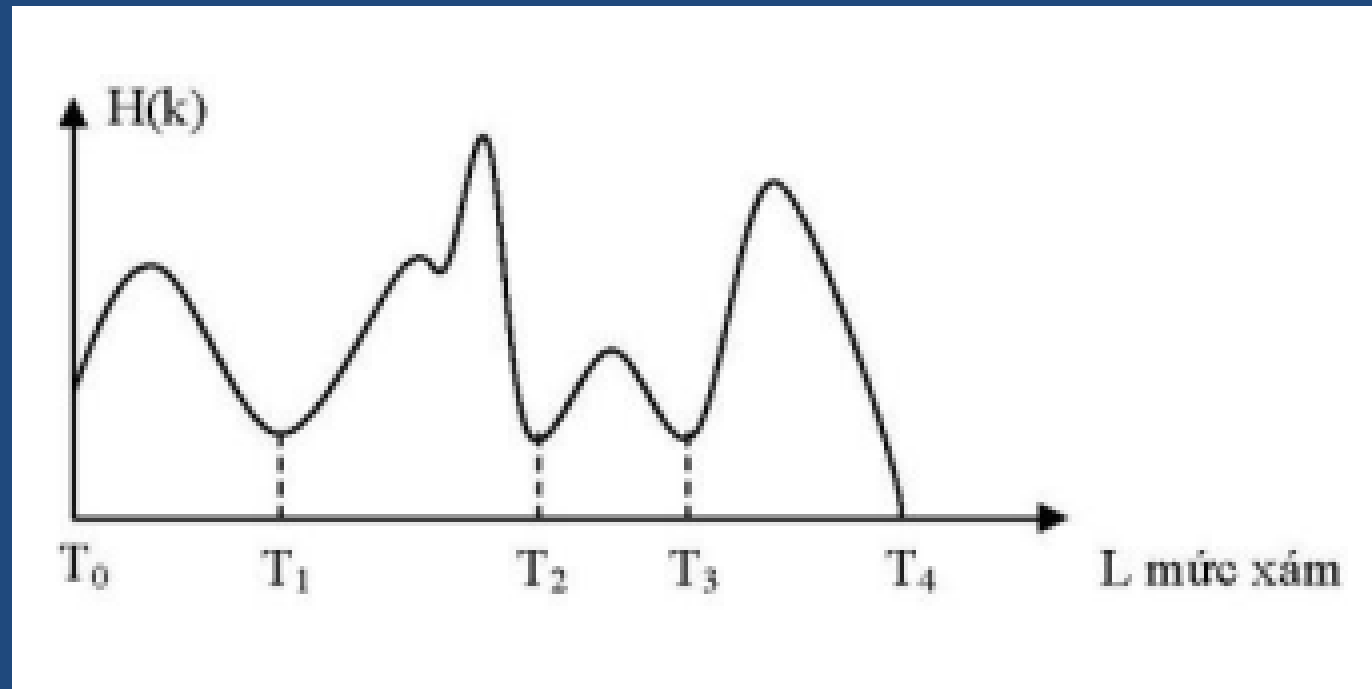


Gray-Level Thresholding



Thresholds

Gray-Level Thresholding



Double thresholding

Here we choose two values T_1 and T_2 and apply a thresholding operation as:

A pixel becomes $\begin{cases} \text{white if its grey level is between } T_1 \text{ and } T_2, \\ \text{black if its grey level is otherwise.} \end{cases}$

Gray-Level Thresholding

- **Multi-level:**

$$g(x, y) = \begin{cases} a & f(x, y) > T_2 \\ b & T_1 < f(x, y) \leq T_2 \\ c & f(x, y) \leq T_1 \end{cases}$$

CODE

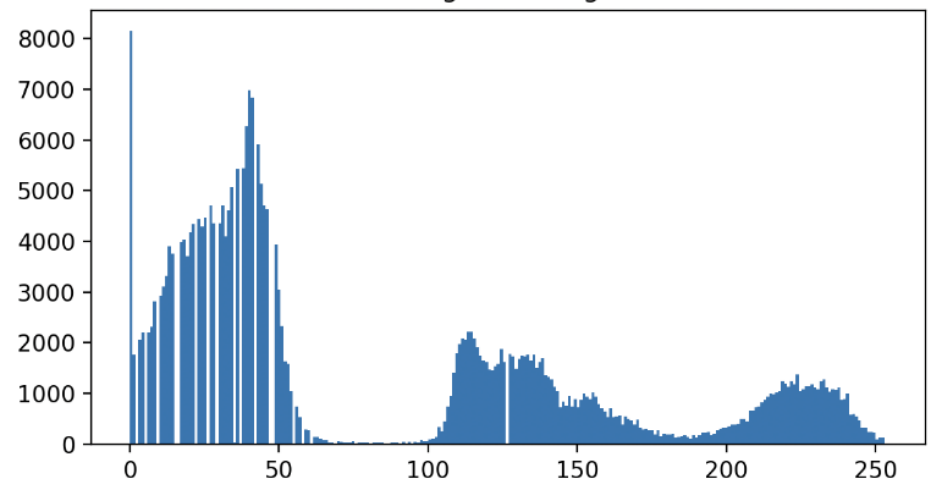
```
def phan_doan_bang_cat_nguong(img,T1,T2): # Định nghĩa hàm phân đoạn bằng cắt ngưỡng
    m, n = img.shape
    img_phan_doan_cat_nguong = np.zeros([m, n])
    for i in range(m):
        for j in range(n):
            if (img[i,j] < T1):
                img_phan_doan_cat_nguong[i,j] = 0
            elif (img[i,j] < T2):
                img_phan_doan_cat_nguong[i,j] = 127
            else:
                img_phan_doan_cat_nguong[i, j] = 255
    return img_phan_doan_cat_nguong
```


Kết quả

Ảnh gốc



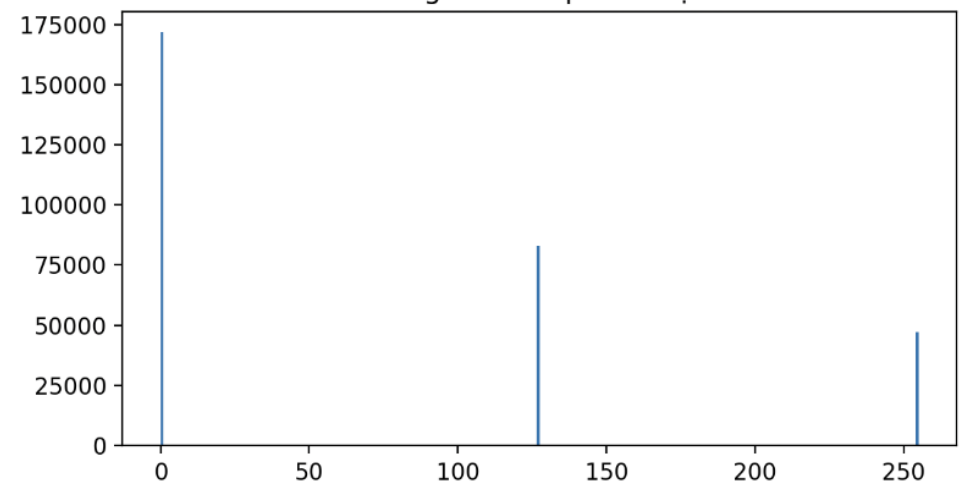
Hitogram ảnh gốc



Ảnh phân đoạn bằng cắt đa ngưỡng



Hitogram ảnh phân đoạn



Thuật toán tìm ngưỡng toàn cục cơ bản

- B1: Chọn giá trị ngưỡng ban đầu T (thường là trung bình mức xám trong ảnh)
- B2: Sử dụng ngưỡng T để phân đoạn ảnh, tạo ra 2 nhóm điểm ảnh (áp dụng công thức (1)):
 - Nhóm $G1$ chứa các pixel với mức xám $> T$
 - Nhóm $G2$ chứa các pixel với mức xám $\leq T$
- B3: Tính mức xám trung bình nhóm $G1$ là m_1 và mức xám trung bình nhóm $G2$ là m_2
- B4: Tính ngưỡng mới:
$$T = \frac{m_1 + m_2}{2} \quad (2)$$
- B5: Lặp lại B2 đến B4 cho đến khi giá trị của T trong các lần lặp tiếp theo nhỏ hơn ΔT

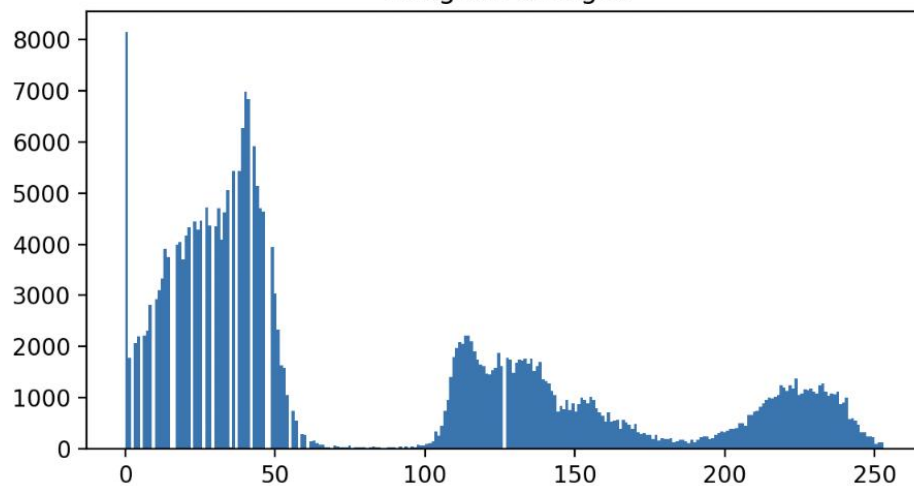
```
def Tim_nguoi_toan_cuc(img): # Định nghĩa hàm tìm ngưỡng
    #Bước 1. Khởi tạo ngưỡng ban đầu t bằng giá trị trung bình mức xám của ảnh
    t=np.mean(img) #initial condition
    g1 = [] # Định nghĩa nhóm g1
    g2 = [] # Định nghĩa nhóm g2
    m,n = img.shape
    # Lặp để tính ngưỡng
    while (True):
        # Bước 2. Tạo nhóm g1,g2 dựa trên ngưỡng t
        for i in range(m):
            for j in range(n):
                if (img[i,j] < t):
                    g1.append(img[i,j])
                else:
                    g2.append(img[i,j])
        # Bước 3. Tính trung bình mức xám trong vùng g1,g2
        m1 = np.mean(g1)
        m2 = np.mean(g2)
        # Bước 4. Tính lại ngưỡng t có giá trị mới
        t_new = ((m1 + m2)/2)
        # Tính delta t để làm điều kiện thoát vòng lặp
        delta_t = abs(t - t_new)
        t = t_new;
        if(delta_t < 1):
            break
    print("Ngưỡng tìm được: ", t)
    return t
```

Kết quả

Ảnh gốc



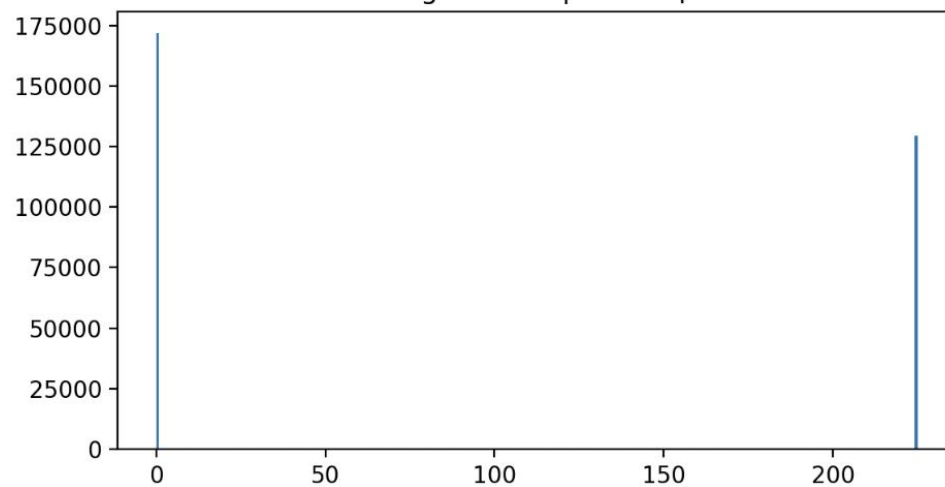
Hitogram ảnh gốc



Ảnh phân đoạn dựa vào tìm ngưỡng cơ bản



Hitogram ảnh phân đoạn



BÀI TẬP

: Cho ảnh số được biểu diễn dạng

$$I = \begin{bmatrix} 1 & 2 & 1 & 1 & 2 \\ 1 & 5 & 5 & 7 & 3 \\ 11 & 5 & 5 & 7 & 3 \\ 1 & 5 & 7 & 7 & 6 \\ 1 & 8 & 7 & 6 & 8 \end{bmatrix}$$

Vẽ lược đồ xám của ảnh.

Tìm ngưỡng theo cách tìm tự động.

BÀI TẬP

: Cho ảnh số được biểu diễn dạng

1	4	7	10	7	4	1
4	12	26	33	26	12	4
7	26	55	71	55	26	7
10	33	71	91	71	33	10
7	26	55	71	55	26	7
4	12	26	33	26	12	4
1	4	7	10	7	4	1

Vẽ lược đồ xám của ảnh.

Tìm ngưỡng theo cách tìm tự động.

BÀI TẬP

Cho ảnh số như hình vẽ

1	1	6	6	6	5	5	5
1	1	1	6	6	5	5	5
1	1	1	6	6	8	5	5
2	2	6	6	6	8	8	8
2	2	7	7	8	8	8	8
2	7	7	7	4	4	4	8
7	7	7	7	4	4	4	4

- Tìm ngưỡng ảnh theo phương pháp tìm ngưỡng tự động.
- Hãy làm nổi biên dùng toán tử Roberts.
- Hãy làm nổi biên dùng toán tử Prewitt.
- Hãy làm nổi biên dùng toán tử Sobel.

Thuật toán tìm ngưỡng Otsu

Là phương pháp tìm ngưỡng dựa trên Histogram ảnh

1. Tính lược đồ xám chuẩn hóa của ảnh đầu vào. Tính các giá trị $p_i = n_i / MN$ ($i=0, 1, \dots, L-1$).
2. Tính tổng tích lũy $P_1(k)$ với $k=0, \dots, L-1$ theo công thức: $P_1(k) = \sum_{i=0}^k p_i$
3. Tính trung bình tích lũy đến mức k là $m(k)$: $m(k) = \sum_{i=0}^k i \cdot p_i$
4. Tính trung bình cường độ toàn cục m_G :
(cường độ trung bình của toàn bộ ảnh). $m_G = \sum_{i=0}^{L-1} i \cdot p_i$
5. Tính phương sai giữa các nhóm $\sigma_B^2(k)$

$$\sigma_B^2(k) = \frac{[m_G P_1(k) - m(k)]^2}{P_1(k)[1 - P_1(k)]}$$

6. Ngưỡng Otsu k^* là giá trị k thỏa mãn điều kiện $\sigma_B^2(k)$ đạt cực đại.
Nếu cực đại không duy nhất thì k^* được tính là trung bình các giá trị k tương ứng với các cực đại tìm thấy được.

Otsu – Ví dụ

0	1	2	3	4	5
0	0	1	2	3	4
0	0	0	1	2	3
0	0	0	0	1	2
0	0	0	0	0	1

1. Tính lược đồ xám chuẩn hóa của ảnh đầu vào. Tính các giá trị $p_i = n_i / MN$ ($i=0,1,\dots, L-1$).
2. Tính tổng tích lũy $P_1(k)$ với $k=0,\dots,L-1$ theo công thức: $P_1(k) = \sum_{i=0}^k p_i$
3. Tính trung bình tích lũy đến mức k là $m(k)$: $m(k) = \sum_{i=0}^k i \cdot p_i$
4. Tính trung bình cường độ toàn cục m_G :
(cường độ trung bình của toàn bộ ảnh). $m_G = \sum_{i=0}^{L-1} i \cdot p_i$
5. Tính phương sai giữa các nhóm $\sigma_B^2(k)$

$$\sigma_B^2(k) = \frac{[m_G P_1(k) - m(k)]^2}{P_1(k)[1 - P_1(k)]}$$

i	p_i	$P_i(k)$	$m(k)$	$m(G)$	$\sigma_B^2(k)$
0	15/30	0.5000	0.0000	1.1667	1.36
1	5/30	0.6667	0.1667	1.1667	1.68
2	4/30	0.8000	0.4333	1.1667	1.56
3	3/30	0.9000	0.7333	1.1667	1.11
4	2/30	0.9667	1.0000	1.1667	0.51
5	1/30	1.0000	1.1667	1.1667	0.00

• Như vậy ngưỡng $T = 1$ với $\sigma_B^2(k) = 1.68$

Otsu's Binarization

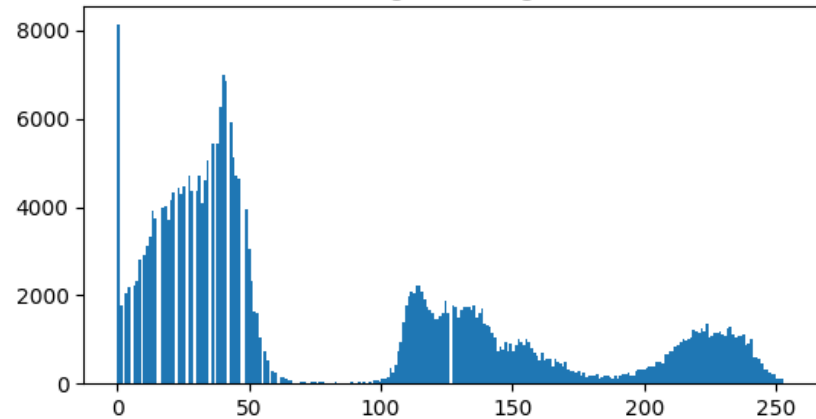
```
# Otsu's thresholding
```

```
ret2,th2 = cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
```

Ảnh gốc



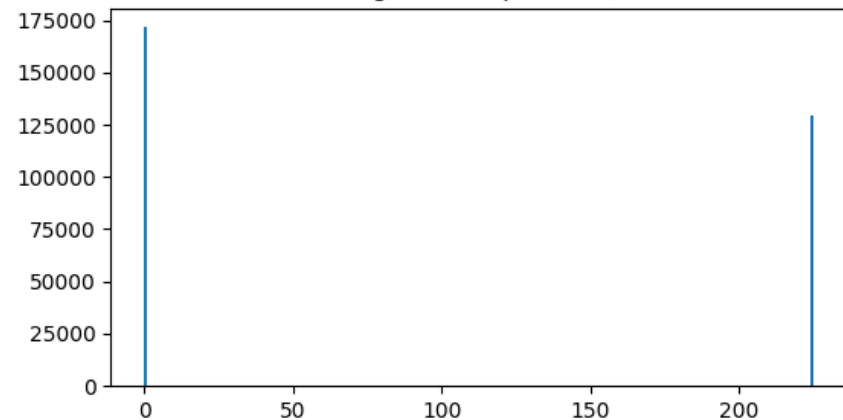
Hitogram ảnh gốc



Ảnh phân đoạn dựa vào tìm ngưỡng Otsu



Hitogram ảnh phân đoạn

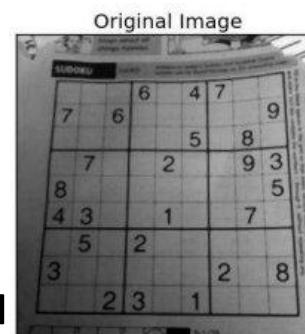


Gray-Level Thresholding

- Phụ thuộc chủ quan
→ phân tích histogram
- Dễ bị ảnh hưởng bởi nhiễu
- Ảnh hưởng bởi thay đổi độ sáng

Adaptive Thresholding

- **Adaptive Thresholding:** là một kỹ thuật xử lý ảnh nhị phân linh hoạt, trong đó ngưỡng không cố định trên toàn bộ ảnh mà thay đổi tùy thuộc vào các đặc tính cục bộ của từng vùng ảnh.
- Tại sao cần Adaptive Thresholding?
 - Ảnh có độ sáng không đồng đều
 - Ảnh có nhiều vùng sáng tối khác nhau



Adaptive Thresholding

Các thuật toán Adaptive Thresholding phổ biến

1. Mean C-based Thresholding:

Ý tưởng: Tính giá trị trung bình của một vùng lân cận xung quanh mỗi pixel, sau đó sử dụng giá trị trung bình này trừ đi hằng số C làm ngưỡng cho pixel đó.

2. Gaussian Weighted Average Thresholding:

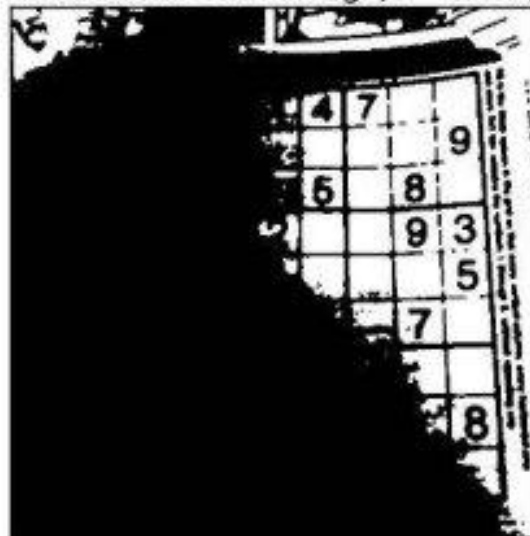
Ý tưởng: Tính giá trị trung bình có trọng số của một vùng lân cận của mỗi pixel, trong đó các pixel gần hơn được gán trọng số lớn hơn.

Adaptive Thresholding

Original Image



Global Thresholding ($\nu = 127$)



Adaptive Mean Thresholding



Adaptive Gaussian Thresholding



Adaptive Thresholding

```
dst = cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType,
    blockSize, C)
```

Example:

```
dst = cv.adaptiveThreshold(img,255,cv.ADAPTIVE_THRESH_MEAN_C,\n    cv.THRESH_BINARY,11,2)
```

```
dst1 = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
cv2.THRESH_BINARY, 11, 2)
```



Bài tập

Cho ảnh I sau:

19	23	37	43	43
10	19	23	37	43
10	10	19	23	37
10	10	10	19	23
9	10	10	10	19

- Xác định ngưỡng T theo phương pháp ngưỡng toàn cục cơ bản (isodata).
- Xác định ngưỡng T_1 theo phương pháp Otsu.
- Thực hiện phân vùng ảnh I với ngưỡng T , T_1 xác định được

Kiểm tra

Bài 1: Cho ảnh I sau:

1	4	1	1	2	3	2	3
3	2	4	2	5	2	6	2
2	1	8	2	5	2	5	6
2	5	2	4	7	9	1	4
2	2	3	0	0	1	2	1
1	5	7	1	2	4	5	6

- Xác định ngưỡng T theo phương pháp ngưỡng toàn cục cơ bản (isodata).
- Xác định ngưỡng T_1 theo phương pháp Otsu.
- Thực hiện phân vùng ảnh I với ngưỡng T, T_1 xác định được